

Java Programming

Constructors — A Complete Student Guide

Understanding Object Initialisation in Java

1. Introduction to Constructors

A **constructor** is a special method in Java that is automatically called when an object is created using the **new** keyword. Its primary job is to **initialise** the fields (instance variables) of the newly created object.

Think of a constructor as the 'birth certificate' of an object — it sets up everything the object needs the moment it comes into existence.

1.1 Why Do We Need Constructors?

- Guarantee that every object starts in a valid state.
- Avoid manually setting every field after object creation.
- Enforce required data at the time of object creation.
- Enable different initialisation paths through overloading.

📌 **Note** Constructors are NOT methods. They have no return type and share the exact name of the class.

2. Constructor Syntax

A constructor follows this general syntax:

```
class ClassName {  
    // Instance variables  
    DataType field1;  
    DataType field2;  
  
    // Constructor  
    ClassName(DataType param1, DataType param2) {  
        this.field1 = param1;  
        this.field2 = param2;  
    }  
}
```

Key Rules

- The constructor name **MUST** match the class name exactly (case-sensitive).
- Constructors have **NO** return type — not even void.
- They can have any access modifier: public, protected, private, or package-private.
- A class can have multiple constructors (constructor overloading).

3. Types of Constructors

3.1 Default Constructor (No-Argument Constructor)

A default constructor takes **no parameters**. If you do not write any constructor, Java **automatically provides one**. As soon as you write any constructor yourself, Java's automatic default constructor disappears.

```
class Student {
    String name;
    int    age;

    // Default constructor
    Student() {
        name = "Unknown";
        age  = 0;
    }
}

// Usage
Student s = new Student(); // calls default constructor
System.out.println(s.name); // Output: Unknown
```

 **Note** Java's compiler-generated default constructor sets all numeric fields to 0, booleans to false, and object references to null.

3.2 Parameterised Constructor

A parameterised constructor **accepts arguments** that are used to initialise the object's fields with meaningful values at the time of creation.

```
class Student {
    String name;
    int    age;
    double gpa;

    // Parameterised constructor
    Student(String name, int age, double gpa) {
        this.name = name; // 'this' refers to the current object
        this.age  = age;
        this.gpa  = gpa;
    }

    void display() {
        System.out.println("Name: " + name + ", Age: " + age + ", GPA: " + gpa);
    }
}

// Usage
Student s1 = new Student("Alice", 20, 3.8);
```

```
Student s2 = new Student("Bob", 22, 3.5);
s1.display(); // Name: Alice, Age: 20, GPA: 3.8
s2.display(); // Name: Bob, Age: 22, GPA: 3.5
```

3.3 Copy Constructor

A copy constructor creates a new object as an **exact copy** of an existing object. Java does not provide one automatically — you must write it yourself.

```
class Student {
    String name;
    int age;

    // Parameterised constructor
    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Copy constructor
    Student(Student other) {
        this.name = other.name;
        this.age = other.age;
    }
}

// Usage
Student original = new Student("Alice", 20);
Student copy = new Student(original); // copy constructor
System.out.println(copy.name); // Alice
```

Note A copy constructor gives you a NEW object with the same data. Changing the copy does NOT affect the original — unlike a simple assignment (`Student copy = original`) which only copies the reference.

4. Constructor Overloading

Java allows multiple constructors in the same class as long as each has a different parameter list (different number or types of parameters). This is called **constructor overloading**.

```
class Rectangle {
    double width;
    double height;

    // Constructor 1 — no args (square with side 1)
    Rectangle() {
```

```

        this.width  = 1.0;
        this.height = 1.0;
    }

    // Constructor 2 – one arg (square)
    Rectangle(double side) {
        this.width  = side;
        this.height = side;
    }

    // Constructor 3 – two args (full rectangle)
    Rectangle(double width, double height) {
        this.width  = width;
        this.height = height;
    }

    double area() { return width * height; }
}

// Usage
Rectangle r1 = new Rectangle();           // 1x1
Rectangle r2 = new Rectangle(5);          // 5x5
Rectangle r3 = new Rectangle(4, 7);        // 4x7
System.out.println(r3.area());            // 28.0

```

5. The this Keyword in Constructors

5.1 Resolving Name Conflicts

When a constructor parameter has the same name as an instance variable, **this.** distinguishes the instance variable from the parameter.

```

class Circle {
    double radius;

    Circle(double radius) {           // parameter 'radius' shadows field 'radius'
        this.radius = radius;         // this.radius → field | radius → parameter
    }
}

```

5.2 Constructor Chaining with this()

One constructor can call another constructor in the same class using **this()**. This eliminates duplicate code and must be the **FIRST statement** in the constructor.

```

class Employee {
    String name;

```

```

    String department;
    double salary;

    Employee(String name) {
        this(name, "General", 30000); // calls 3-arg constructor
    }

    Employee(String name, String department) {
        this(name, department, 30000); // calls 3-arg constructor
    }

    Employee(String name, String department, double salary) {
        this.name      = name;
        this.department = department;
        this.salary     = salary;
    }
}

```

6. Constructor Inheritance and super()

When a class extends another class, the child class constructor must call the parent class constructor. Java inserts **super()** automatically if you don't, but only if the parent has a no-arg constructor. If the parent only has parameterised constructors, you must call **super(...)** explicitly.

```

class Animal {
    String name;

    Animal(String name) {
        this.name = name;
        System.out.println("Animal constructor called: " + name);
    }
}

class Dog extends Animal {
    String breed;

    Dog(String name, String breed) {
        super(name); // MUST be first statement
        this.breed = breed;
        System.out.println("Dog constructor called: " + breed);
    }
}

// Usage
Dog d = new Dog("Rex", "Labrador");
// Output:
// Animal constructor called: Rex
// Dog constructor called: Labrador

```

 **Note** `super()` must always be the first statement in a child constructor. You cannot have both `super()` and `this()` in the same constructor.

7. Access Modifiers on Constructors

Constructors can have access modifiers that control which code can create instances of a class.

Modifier	Who Can Instantiate	Common Use Case
<code>public</code>	Any class anywhere	Most common — standard objects
<code>protected</code>	Same package + subclasses	Abstract base classes
<code>(none)</code>	Same package only	Utility classes within a package
<code>private</code>	Only inside the class itself	Singleton pattern

Private Constructor — Singleton Pattern

```
class DatabaseConnection {
    private static DatabaseConnection instance = null;

    private DatabaseConnection() {           // private — cannot use 'new'
outside
        System.out.println("DB connected");
    }

    public static DatabaseConnection getInstance() {
        if (instance == null) {
            instance = new DatabaseConnection();
        }
        return instance;
    }
}

// Usage — always returns the SAME instance
DatabaseConnection db = DatabaseConnection.getInstance();
```

8. Common Mistakes & How to Fix Them

Mistake 1 — Adding a Return Type

```
// WRONG — this is a method, not a constructor!
class Student {
    void Student() {    // <-- void makes it a regular method
        ...
    }
}
```

```

    }
}

// CORRECT
class Student {
    Student() {          // no return type
        ...
    }
}

```

Mistake 2 — Mismatched Constructor Name

```

// WRONG
class Student {
    student() { ... } // lowercase 's' - won't compile
}

// CORRECT
class Student {
    Student() { ... } // matches class name exactly
}

```

Mistake 3 — Forgetting super() When Needed

```

class Vehicle {
    int speed;
    Vehicle(int speed) { this.speed = speed; }
}

// WRONG - Vehicle has no no-arg constructor
class Car extends Vehicle {
    Car() { } // compiler error!
}

// CORRECT
class Car extends Vehicle {
    Car(int speed) { super(speed); }
}

```

Mistake 4 — Recursive Constructor Chaining

```

// WRONG - causes StackOverflowError
class A {
    A() { this(1); }
    A(int x) { this(); } // circular!
}

```


9. Constructor vs Method — Key Differences

Feature	Constructor	Method
Name	Same as class name	Any valid identifier
Return Type	None (not even void)	Must declare one (void allowed)
Called By	Automatically on new	Explicitly by the programmer
Inheritance	Not inherited	Inherited (unless private)
Purpose	Object initialisation	Define object behaviour

10. Full Worked Example — BankAccount Class

The following example demonstrates all constructor concepts together in a realistic scenario.

```
public class BankAccount {

    // Instance variables
    private String accountNumber;
    private String holderName;
    private double balance;
    private String accountType;

    // — Constructor 1: minimal — account number only —————
    public BankAccount(String accountNumber) {
        this(accountNumber, "Unknown", 0.0, "Savings");
    }

    // — Constructor 2: account number + holder name —————
    public BankAccount(String accountNumber, String holderName) {
        this(accountNumber, holderName, 0.0, "Savings");
    }

    // — Constructor 3: full initialisation —————
    public BankAccount(String accountNumber, String holderName,
                        double balance, String accountType) {
        this.accountNumber = accountNumber;
        this.holderName     = holderName;
        this.balance        = balance;
        this.accountType    = accountType;
    }

    // — Copy constructor —————
    public BankAccount(BankAccount other) {
        this(other.accountNumber, other.holderName,
              other.balance, other.accountType);
    }

    // — Behaviour methods —————
```

```

    public void deposit(double amount) { balance += amount; }
    public void withdraw(double amount) {
        if (amount <= balance) balance -= amount;
        else System.out.println("Insufficient funds");
    }
    public void printDetails() {
        System.out.printf("Account: %s | Holder: %s | Type: %s | Balance:
%.2f%n",
                                accountNumber, holderName, accountType, balance);
    }

    // — main method —————
    public static void main(String[] args) {
        BankAccount acc1 = new BankAccount("ACC001");
        BankAccount acc2 = new BankAccount("ACC002", "Alice");
        BankAccount acc3 = new BankAccount("ACC003", "Bob", 5000.0,
"Current");
        BankAccount acc4 = new BankAccount(acc3);           // copy

        acc2.deposit(1500);
        acc3.withdraw(200);

        acc1.printDetails();
        acc2.printDetails();
        acc3.printDetails();
        acc4.printDetails();
    }
}

```

Expected Output:

```

Account: ACC001 | Holder: Unknown | Type: Savings | Balance: 0.00
Account: ACC002 | Holder: Alice   | Type: Savings | Balance: 1500.00
Account: ACC003 | Holder: Bob     | Type: Current  | Balance: 4800.00
Account: ACC003 | Holder: Bob     | Type: Current  | Balance: 5000.00

```

11. Practice Exercises

Complete the following exercises to reinforce your understanding of constructors.

Exercise 1 — Basic Constructor

Create a class called `Book` with the following fields: `title` (`String`), `author` (`String`), `pages` (`int`), `price` (`double`).

- Write a no-argument constructor that sets default values.
- Write a parameterised constructor that accepts all four fields.
- Write a `display()` method and test both constructors in `main()`.

Exercise 2 — Constructor Overloading

Create a class called Pizza with fields: size (String), toppings (int), price (double).

- Constructor 1: size only (toppings = 0, price = 5.00).
- Constructor 2: size + toppings (price = 5.00 + 1.50 per topping).
- Constructor 3: all three fields.
- Create three Pizza objects and print their details.

Exercise 3 — Inheritance & super()

Create a Shape base class with a colour field, then extend it with Circle and Rectangle classes.

- Shape constructor: takes colour.
- Circle constructor: takes colour + radius, calls super(colour).
- Rectangle constructor: takes colour + width + height, calls super(colour).
- Add an area() method to both Circle and Rectangle and test them.

Exercise 4 — Copy Constructor

Add a copy constructor to your Book class from Exercise 1.

- Create a Book object, copy it, change a field of the copy, and verify the original is unchanged.
- Explain in a comment why this matters vs. simple assignment.

12. Quick Reference Summary

Concept	Key Point
Default Constructor	No parameters. Java provides one if you write none.
Parameterised Constructor	Accepts arguments to initialise fields with values.
Copy Constructor	Takes an object of the same class; duplicates its state.
this keyword	Refers to the current object; resolves name conflicts.
this() call	Chains to another constructor; MUST be first statement.
super() call	Calls parent constructor; MUST be first statement.
Overloading	Multiple constructors with different parameter lists.